

Product Review & Rating Platform

Yucen Wang

Department of Information Technology, Monroe University, King Graduate School

KG630-154W: Database Systems

Professor Deepthi Umesha

April 13, 2026

Product Review & Rating Platform

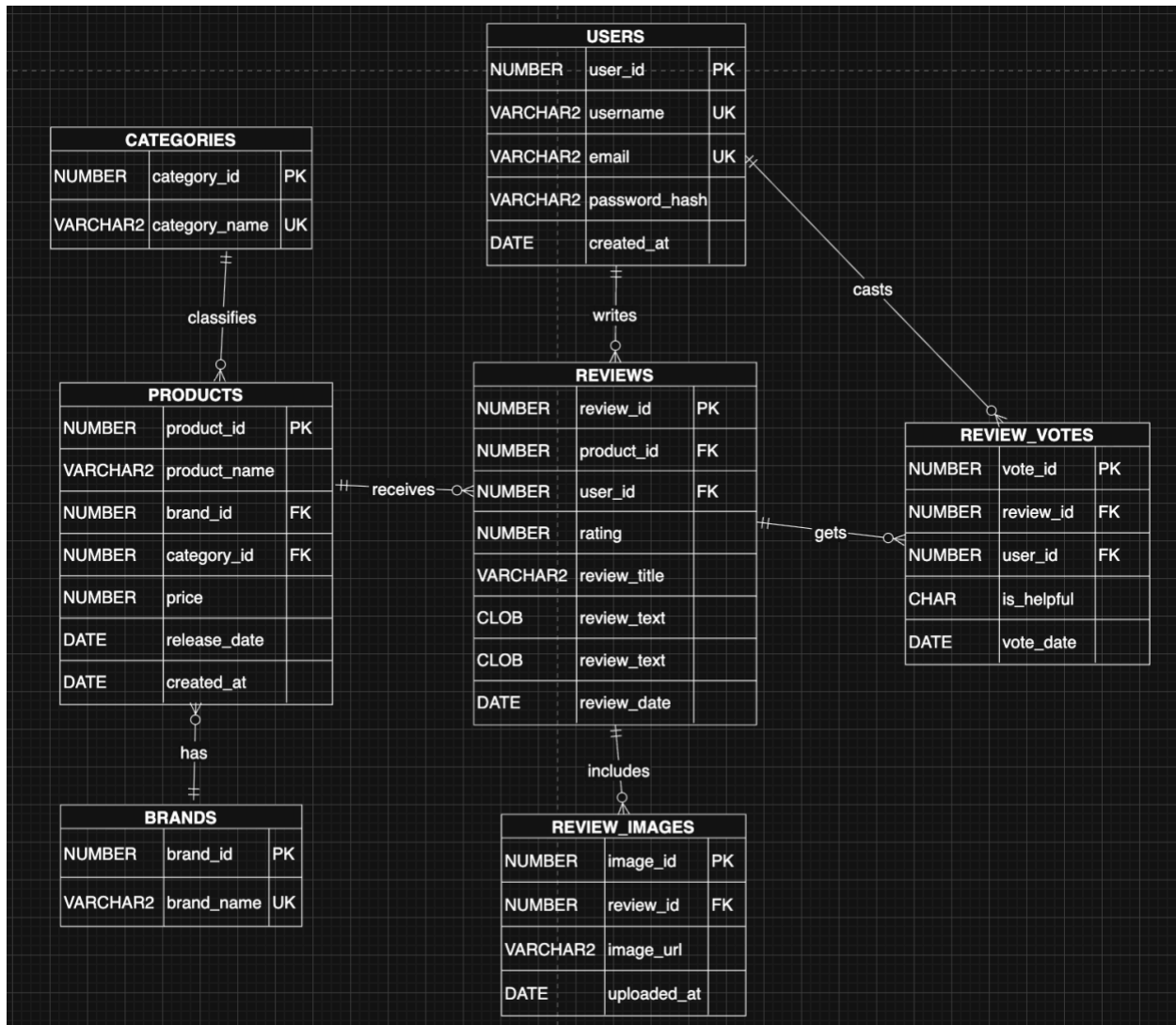
Summary

This project creates a database design for a product review and rating platform. The database includes users, products, brands, categories, reviews, review votes and review images tables. The design focuses on data integrity, normalization to 3NF, and practical review analysis of use cases.

Introduction

Websites like Amazon allow customers to leave product reviews, give rating scores, and vote on whether a review is helpful. I chose this project to model a similar system and show how an Oracle database can store and manage products, users, reviews, ratings, and review votes using database design and sample data.

ER Diagram



There are 7 tables in this database, represented as 7 entities in the ER diagram. A single category or brand can be associated with many products, while each product belongs to only one category and one brand. Each product receives more than one review, but that review only links to the specific product. One user can write many reviews and cast many review votes. Each review can include multiple images and have multiple review votes, whereas one review vote or one review image is linked to one specific review.

Data Modeling

The users table stores reviewer account information. For this table, user_id is the unique primary key. The username and email are set as unique keys to avoid any confusion. Each user can set his or her password and the created time saved in the users table.

	USER_ID	USERNAME	EMAIL	PASSWORD_HASH	CREATED_AT
1	1	alice01	alice@example.com	hash1	14-MAR-26
2	2	bob02	bob@example.com	hash2	14-MAR-26
3	3	carol03	carol@example.com	hash3	14-MAR-26

The brands table records the product brand. Brand_id is the primary key for the table and brand_name should be unique.

	BRAND_ID	BRAND_NAME
1	1	Apple
2	2	Samsung
3	3	Sony

The categories table records products' categories. Category_id is the primary key for this table and category_name should be unique.

	CATEGORY_ID	CATEGORY_NAME
1	2	Headphones
2	3	Laptops
3	1	Smartphones

The products table contains all the products listed on the platform. Product_id is the primary key. Brand_id is a foreign key that can connect with the brands table. Category_id is the second foreign key that connects with the categories table.

	PRODUCT_ID	PRODUCT_NAME	BRAND_ID	CATEGORY_ID	PRICE	RELEASE_DATE	CREATED_AT
1	1	iPhone 15	1	1	999.99	22-SEP-23	14-MAR-26
2	2	Galaxy S24	2	1	899.99	17-JAN-24	14-MAR-26
3	3	WH-1000XM5	3	2	399.99	12-MAY-22	14-MAR-26

The reviews table store user feedback and rating scores. Review_id is the primary key, representing each review of products. Product_id and user_id are foreign keys that connect with the user who writes the review of this product. The table also stores the rating, review title, content and review date.

	REVIEW_ID	PRODUCT_ID	USER_ID	RATING	REVIEW_TITLE	REVIEW_TEXT	REVIEW_DATE
1	1	1	1	5	Excellent phone	Great battery life and camera quality.	14-MAR-26
2	2	1	2	4	Very good but expensive	Performance is excellent, but price is high.	14-MAR-26
3	3	2	3	4	Solid Android option	Display and speed are impressive.	14-MAR-26
4	4	3	2	5	Best headphones	Noise cancellation is amazing.	14-MAR-26

The review_votes table captures users' feedback on review usefulness. Vote_id is the primary key to represent each vote. Review_id and User_id are foreign key that connect with the user who votes for the specific review.

	VOTE_ID	REVIEW_ID	USER_ID	IS_HELPFUL	VOTE_DATE
1	1	1	2	Y	14-MAR-26
2	2	1	3	Y	14-MAR-26
3	3	2	1	Y	14-MAR-26

When the reviews include an image, the review_images table records the images' URL link and the uploaded date. Image_id is the primary key and review_id is the foreign key that connects to the reviews table.

	IMAGE_ID	REVIEW_ID	IMAGE_URL	UPLOADED_AT
1	1	1	https://example.com/review1_img1.jpg	14-MAR-26
2	2	4	https://example.com/review4_img1.jpg	14-MAR-26

Database Design

All tables in my database are normalized to Third Normal Form (3NF) so the data stays organized. I used normalization to make the database easier to manage.

1. In First Normal Form (1NF), each cell in the table contains only one value, and there are no repeating groups. Each row is unique, and each column contains one type of data only.
2. In Second Normal Form (2NF), each table has a primary key, and all the non-key columns depend on the whole primary key. Because of this, there are no partial dependencies in the database.
3. In Third Normal Form (3NF), the non-key attributes depend only on the primary key and not on other non-key attributes. This means there are no transitive dependencies. Each table in my design focuses on one main type of information, and the relationships between tables are connected using primary keys and foreign keys.

Brands and categories tables are stored in separate tables instead of being stacked directly into the products table. I did this to avoid repeating the same brand name or category name for different products. If multiple products belong to Apple or are in the same category, that information only needs to be stored once, which makes the design cleaner and easier to update.

I also kept review votes in a separate table from reviews because they represent a different kind of data. One review can receive many votes from different users, so putting vote information inside the review table would not make sense. Keeping it separate makes the structure more organized and matches how a real system would work.

I added unique constraints to make sure the data follows important rules. For example, usernames and emails should be unique so each user account is different. I also used unique constraints to make sure one user cannot review the same product more than once, and one user

cannot vote on the same review multiple times. These constraints help keep the data accurate and make the database work more like a real product review platform such as Amazon.

Analysis

When we left join the products table with the reviews table, and group by product_id and product_name, we can calculate the average rating per product based on the reviews. From the screenshot, we can see product WH-1000XM5 has the highest rating based on 1 review.

```
-- Average rating per product
SELECT p.product_id,
       p.product_name,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(r.review_id) AS total_reviews
FROM products p
LEFT JOIN reviews r
      ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
ORDER BY avg_rating DESC NULLS LAST;
```

Query Result x

SQL | All Rows Fetched: 3 in 0.173 seconds

	PRODUCT_ID	PRODUCT_NAME	AVG_RATING	TOTAL_REVI...
1	3	WH-1000XM5	5	1
2	1	iPhone 15	4.5	2
3	2	Galaxy S24	4	1

If we want to get all the reviews with their correlated user and product, we can use inner joins on the reviews table, the users table and the products table.

```

-- All reviews with username and product
SELECT u.username,
       p.product_name,
       r.rating,
       r.review_title,
       r.review_date
FROM reviews r
JOIN users u
    ON r.user_id = u.user_id
JOIN products p
    ON r.product_id = p.product_id
ORDER BY r.review_date DESC;

```

Query Result x

SQL | All Rows Fetched: 4 in 0.118 seconds

	USERNAME	PRODUCT_NAME	RATING	REVIEW_TITLE	REVIEW_DATE
1	alice01	iPhone 15	5	Excellent phone	14-MAR-26
2	carol03	Galaxy S24	4	Solid Android option	14-MAR-26
3	bob02	WH-1000XM5	5	Best headphones	14-MAR-26
4	bob02	iPhone 15	4	Very good but expensive	14-MAR-26

Using reviews table to left join the review_votes table, we can connect the information and learn whether a review is helpful or not through the votes. From the screenshot, the “Excellent phone” review is helpful to the users.

```

-- Helpful vote counts per review
SELECT r.review_id,
       r.review_title,
       SUM(CASE WHEN rv.is_helpful = 'Y' THEN 1 ELSE 0 END) AS helpful_votes,
       SUM(CASE WHEN rv.is_helpful = 'N' THEN 1 ELSE 0 END) AS not_helpful_votes
FROM reviews r
LEFT JOIN review_votes rv
    ON r.review_id = rv.review_id
GROUP BY r.review_id, r.review_title
ORDER BY helpful_votes DESC;

```

Query Result x

SQL | All Rows Fetched: 4 in 0.485 seconds

	REVIEW_ID	REVIEW_TITLE	HELPFUL_VOTES	NOT_HELPFUL_VOTES
1	1	Excellent phone	2	0
2	2	Very good but expensive	1	0
3	3	Solid Android option	0	0
4	4	Best headphones	0	0

By inner joining the products table, categories table and reviews table, we can see the average rating for each product and its category_name.

```
-- Top-rated products in a category
SELECT c.category_name,
       p.product_name,
       ROUND(AVG(r.rating), 2) AS avg_rating
FROM products p
JOIN categories c
    ON p.category_id = c.category_id
JOIN reviews r
    ON p.product_id = r.product_id
GROUP BY c.category_name, p.product_name
ORDER BY c.category_name, avg_rating DESC;
```

Query Result x

SQL | All Rows Fetched: 3 in 0.707 seconds

	CATEGORY_NAME	PRODUCT_NAME	AVG_RATING
1	Headphones	WH-1000XM5	5
2	Smartphones	iPhone 15	4.5
3	Smartphones	Galaxy S24	4

If we want to check how many reviews each user has written, we can use users table to left join reviews table and count the number of review_id. By ordering the total_reviews, we can check each user with its number of reviews in descending order. In the screenshot, we can see “bob02” has the most number of reviews.

```
-- 5. Number of reviews written by each user
SELECT u.user_id,
       u.username,
       COUNT(r.review_id) AS total_reviews
FROM users u
LEFT JOIN reviews r
    ON u.user_id = r.user_id
GROUP BY u.user_id, u.username
ORDER BY total_reviews DESC;
```

Query Result x

SQL | All Rows Fetched: 3 in 0.12 seconds

	USER_ID	USERNAME	TOTAL_REVIEWS
1	2	bob02	2
2	3	carol03	1
3	1	alice01	1

We can also check whether any products have not received any reviews by joining products table with reviews table. Based on the screenshot, no rows are returned, which means every product has at least one review.

```
-- 6. Products with no reviews yet
SELECT p.product_id,
       p.product_name,
       r.review_id
FROM products p
LEFT JOIN reviews r
  ON p.product_id = r.product_id
WHERE r.review_id IS NULL
ORDER BY p.product_name;
```

Query Result x

SQL | All Rows Fetched: 0 in 0.053 seconds

PRODUCT_ID	PRODUCT_NAME	REVIEW_ID
------------	--------------	-----------

By inner joining reviews table and review_images table, we can see which reviews have images.

```
-- 7. Reviews that have images
SELECT r.review_id,
       r.review_title,
       ri.image_url
FROM reviews r
JOIN review_images ri
  ON r.review_id = ri.review_id
ORDER BY r.review_id;
```

Query Result x

SQL | All Rows Fetched: 2 in 0.019 seconds

	REVIEW_ID	REVIEW_TITLE	IMAGE_URL
1	1	Excellent phone	https://example.com/review1_img1.jpg
2	4	Best headphones	https://example.com/review4_img1.jpg

By inner joining review_votes table and users table, we can check each users who has voted on reviews.

```

-- 8. Users who voted on reviews
SELECT u.username,
       rv.review_id,
       rv.is_helpful,
       rv.vote_date
FROM review_votes rv
JOIN users u
  ON rv.user_id = u.user_id
ORDER BY rv.vote_date DESC;

```

Query Result				
All Rows Fetched: 3 in 0.09 seconds				
	USERNAME	REVIEW_ID	IS_HELPFUL	VOTE_DATE
1	alice01	2	Y	14-MAR-26
2	carol03	1	Y	14-MAR-26
3	bob02	1	Y	14-MAR-26

Conclusion

This project shows how to use an Oracle database to support a product review and rating platform. The database was designed to store products, users, reviews, ratings, votes, and review images in a structured and organized way. By using proper table relationships, constraints, and normalization to 3NF, the design helps keep the data accurate and easy to manage.

Through this project, I was able to apply database design concepts such as ER modeling, primary keys, foreign keys, normalization, and SQL implementation in a practical way. This project demonstrates how a well-designed database can support real-world business needs and provide a strong foundation for a database-backed application.